Workshop n°2

Intégration de données avec Spark

Version tuteur - Octobre 2026



Sommaire

Sommaire 2

Présentation 4

Mise en place de l'environnement 5

1. Créez l'arborescence suivante 5

2. Copiez ce fichier docker-compose.yml 5

3. Démarrez et vérifiez 6

4. Créez le module de connexion work/ws_spark.py 6

Partie 1 : Prise en main de PySpark 8

Étape 1 – Générer les données 8

Étape 2 – Trier les données par date de naissance 9

Étape 3 – Construire l'email et l'âge 9

Étape 4 – Filtrer les personnes de 20 ans et plus 9

Étape 5 – Exporter les trois fichiers 9

Étape 6 – Vérifier le résultat 10

Questions 10

Partie 2 : intégration des données de ventes dans PostgreSQL 10

Étape 1 – Récupérer les données 10

Étape 2 – Extraire (le E de ETL) 10

Étape 3 – Analyser la qualité des données 11

Étape 4 – Transformer (le T de ETL) 11

Étape 5 – Contrôler avant de charger 12

Étape 6 – Charger (le L de ETL) 12

Étape 7 – Vérifier dans PostgreSQL 12

Questions 12

Partie 3 : modèle en étoile, datalake et ClickHouse 13

Étape 1 – Récupérer les données 13

Étape 2 – Comprendre le besoin métier 13
Étape 3 – Concevoir le schéma décisionnel 13
Étape 4 – Zone bronze : déposer les données brutes 13
Étape 5 – Zone silver : analyser et nettoyer 14
Étape 6 – Zone gold : alimenter les dimensions 14
Étape 7 – Zone gold : construire la table de faits 15
Étape 8 – Vérifier le datalake 15
Étape 9 – Charger l'étoile dans ClickHouse 15
Étape 10 – Répondre au besoin métier 15
Équivalents Talend 16
Aide-mémoire PySpark 17
En cas de problème 17

Présentation

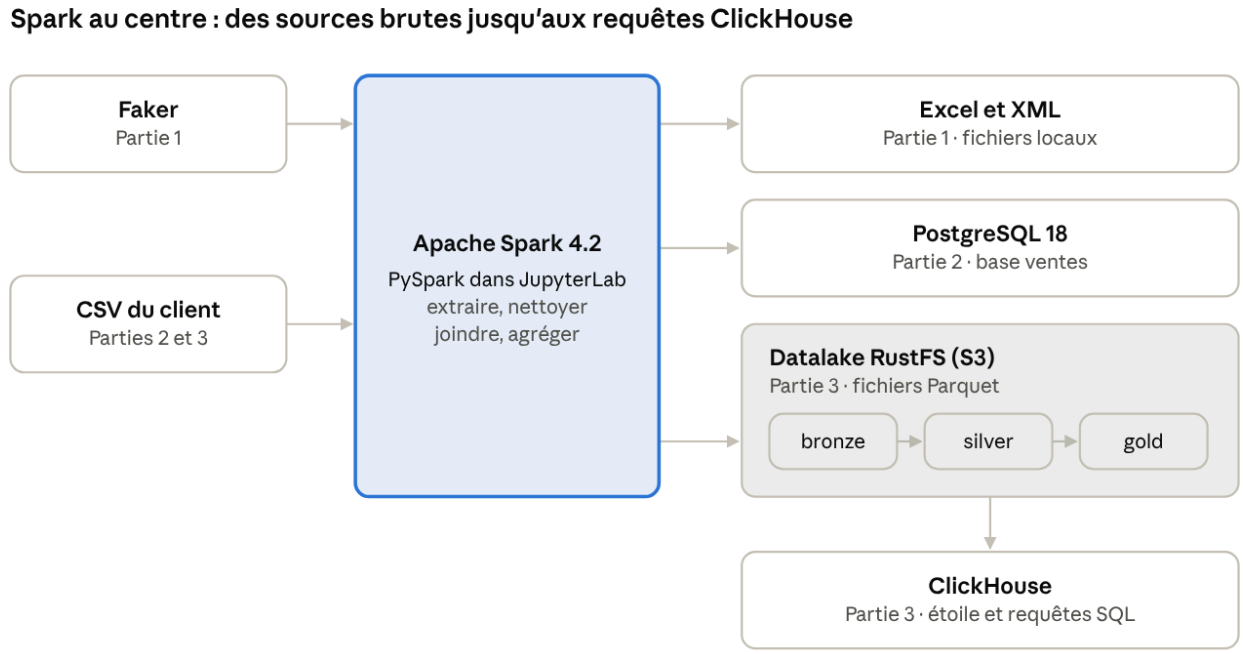
Ce workshop vise à prendre en main le concept d'ETL : extraire des données hétérogènes depuis n'importe quelle source, les transformer, puis les stocker pour les analyser. Vous utiliserez les outils du data engineering actuel : Apache Spark (PySpark), PostgreSQL, un datalake sur stockage objet S3 et l'entrepôt analytique ClickHouse.

Le workshop comporte trois parties :

➡ **Partie 1** : prise en main de PySpark en générant, triant, transformant et exportant de données.

➡ **Partie 2** : intégration des données de ventes d'un client, avec nettoyage, puis chargement dans PostgreSQL.

➡ **Partie 3** : conception d'un modèle décisionnel en étoile, alimentation d'un datalake, puis analyse dans ClickHouse.



Architecture du workshop · 4 services Docker

Spark est votre seul outil de transformation. Chaque partie ajoute une destination, et la partie 3 enchaîne datalake puis ClickHouse.

Prérequis : bases de Python et de SQL, Docker Desktop installé, 8 Go de RAM minimum(16 Go conseillés), environ 6 Go d'espace disque libre.

Documentation utile : API PySpark · Fonctions SQL Spark · SQL ClickHouse

Objectifs pédagogiques :

➡ Comprendre le principe ETL : extraire des données hétérogènes, les transformer, les charger.

➡ Traiter les erreurs classiques : formats de date mixtes, valeurs manquantes, doublons, lignes vides.

➡ Alimenter une base relationnelle, puis concevoir et alimenter un modèle décisionnel en étoile.

➡ Organiser un datalake en zones (bronze, silver, gold) et l'interroger en SQL.

Mise en place de l'environnement

Un seul fichier `docker-compose.yml` démarre les quatre services. Vous travaillerez ensuite uniquement dans JupyterLab.

1. Créez l'arborescence suivante

```
ws-integration/
├── docker-compose.yml
├── data/ # CSV du client (Parties 2 et 3)
└── work/ # vos notebooks + ws_spark.py
```

2. Copiez ce fichier docker-compose.yml

```yaml
services:
  spark:
    image: quay.io/jupyter/pyspark-notebook:spark-4.2.0
    container_name: ws-spark
    ports:
      - "8888:8888" # JupyterLab
      - "4040:4040" # Spark UI
    environment:
      JUPYTER_TOKEN: cesi
    volumes:
      - ./work:/home/jovyan/work
      - ./data:/home/jovyan/data
  postgres:
    image: postgres:18
    container_name: ws-postgres
    environment:
      POSTGRES_USER: etl
      POSTGRES_PASSWORD: etl
      POSTGRES_DB: ventes
    ports:
      - "5433:5432"
    volumes:
      - pgdata:/var/lib/postgresql
  rustfs:
```

```
    image: rustfs/rustfs:1.0.0
    container_name: ws-rustfs
    environment:
      RUSTFS_ACCESS_KEY: rustfsadmin
      RUSTFS_SECRET_KEY: rustfsadmin
      RUSTFS_CONSOLE_ENABLE: "true"
    ports:
      - "9100:9000" # API S3
      - "9101:9001" # Console web
    volumes:
      - s3data:/data
  clickhouse:
    image: clickhouse/clickhouse-server:latest
    container_name: ws-clickhouse
    environment:
      CLICKHOUSE_USER: etl
      CLICKHOUSE_PASSWORD: etl
      CLICKHOUSE_DEFAULT_ACCESS_MANAGEMENT: 1
    ports:
      - "8123:8123" # HTTP + interface SQL web (/play)
    ulimits:
      nofile: { soft: 262144, hard: 262144 }
    volumes:
      - chdata:/var/lib/clickhouse

volumes:
  pgdata:
  s3data:
  chdata:
```

3. Démarrez et vérifiez

1. Dans le dossier `ws-integration`, lancez `docker compose up -d`, puis `docker compose ps` : les quatre conteneurs doivent être `running`.
2. Ouvrez JupyterLab : http://localhost:8888/?token=cesi
3. Ouvrez la console RustFS : http://localhost:9101 (rustfsadmin / rustfsadmin). Créez un bucket nommé `datalake`.
4. Ouvrez ClickHouse : http://localhost:8123/play (etl / etl) et exécutez `SELECT version()`.
5. Testez PostgreSQL : `docker exec -it ws-postgres psql -U etl -d ventes -P pager=off -c "select version();"`

4. Créez le module de connexion work/ws_spark.py

Ce module joue le rôle des connexions du référentiel Talend : tous vos notebooks l'importeront. Avant de le créer, ouvrez un terminal dans JupyterLab et lancez `ls /usr/local/spark/jars | grep hadoop-client-api`. Reportez le numéro de version affiché dans `HADOOP_VERSION`. Dans `work/ws_spark.py` coller :

```
from pyspark.sql import SparkSession

HADOOP_VERSION = "3.5.0"      # Spark 4.2.0 embarque Hadoop 3.5.0 : à
revérifier si l'image change
PG_DRIVER = "org.postgresql:postgresql:42.7.8"

PG_URL = "jdbc:postgresql://postgres:5432/ventes"
PG_PROPS = {"user": "etl", "password": "etl", "driver":
"org.postgresql.Driver"}
LAKE = "s3a://datalake"

# Chemins vus depuis le conteneur Spark. Ils correspondent aux dossiers
# de votre machine montés dans docker-compose.yml :
#   ./data  ->  /home/jovyan/data   (CSV du client)
#   ./work  ->  /home/jovyan/work   (notebooks et fichiers produits)
# "jovyan" est l'utilisateur par défaut des images Jupyter.
DATA = "/home/jovyan/data"
SORTIE = "/home/jovyan/work/sortie"


def get_spark(app="ws-integration"):
    return (
        SparkSession.builder.appName(app)
        .config("spark.jars.packages",
                f"{PG_DRIVER},org.apache.hadoop:hadoop-aws:{HADOOP_VERSION}")
        .config("spark.hadoop.fs.s3a.endpoint", "http://rustfs:9000")
        .config("spark.hadoop.fs.s3a.endpoint.region", "us-east-1")
        .config("spark.hadoop.fs.s3a.access.key", "rustfsadmin")
        .config("spark.hadoop.fs.s3a.secret.key", "rustfsadmin")
        .config("spark.hadoop.fs.s3a.path.style.access", "true")
        .config("spark.hadoop.fs.s3a.connection.ssl.enabled", "false")
        .config("spark.sql.session.timeZone", "Europe/Paris")
        .getOrCreate()
    )
```

Au premier appel, `get_spark()` télécharge ses dépendances. Comptez une à deux minutes.

Partie 1 : Prise en main de PySpark

Vous allez générer un jeu de données, le trier, le transformer, puis l'exporter en trois fichiers. Créez le notebook `work/partie1.ipynb`.

Nous souhaitons obtenir 100 personnes avec :

- un identifiant entier ;
- un nom et un prénom ;
- une date de naissance comprise entre 2000 et 2010 ;
- une ville de naissance ;
- une adresse mail de la forme `prenom.nom@cesi.fr`.

Les données seront triées par date de naissance, puis exportées en trois fichiers : un fichier XML et deux fichiers Excel, dont l'un ne contiendra que les personnes de 20 ans et plus.

Étape 1 – Générer les données

La bibliothèque Faker produit des noms, prénoms et villes réalistes. Installez-la, ainsi qu'openpyxl pour l'export Excel : `pip install -q faker openpyxl`

Créez ensuite la session Spark, générez les lignes avec Faker et transformez-les en DataFrame Spark selon un schéma typé :

```
import datetime as dt
from faker import Faker
from pyspark.sql import functions as F, types as T
from ws_spark import get_spark, SORTIE

spark = get_spark("partie1")
fake = Faker("fr_FR")
Faker.seed(42)

N = 100
rows = [
      (i, fake.last_name(), fake.first_name(),
      fake.date_between(dt.date(2000, 1, 1), dt.date(2010, 12, 31)),
      fake.city())
      for i in range(1, N + 1)
]

schema = T.StructType([
      T.StructField("id", T.IntegerType(), False),
      T.StructField("nom", T.StringType()),
      T.StructField("prenom", T.StringType()),
      T.StructField("date_naissance", T.DateType()),
      T.StructField("ville_naissance", T.StringType()),
])
personnes = spark.createDataFrame(rows, schema)
personnes.printSchema()
personnes.show(5)
```

`printSchema()` affiche la structure du DataFrame et `show()` ses premières lignes. Vérifiez que les types correspondent à ceux demandés.

Étape 2 – Trier les données par date de naissance

```
personnes_triees = personnes.orderBy("date_naissance")
```

Étape 3 – Construire l'email et l'âge

Une simple concaténation prénom + "." + nom produirait des adresses invalides, comme `Hélène.Le Gall@cesi.fr`. On normalise donc chaque partie : passage en minuscules, suppression des accents, puis suppression de tout caractère autre qu'une lettre.

```
def normaliser(col):
    sans_accents = F.translate(F.lower(col),
                               "àâäçéèêëîïôöùûüÿ", "aaaceeeeiioouuuy")
    return F.regexp_replace(sans_accents, r"[^a-z]", "")

sortie = (
    personnes_triees
    .withColumn("email", F.concat(normaliser(F.col("prenom")), F.lit("."),
                                  normaliser(F.col("nom")),
F.lit("@cesi.fr")))
    .withColumn("age", F.floor(F.months_between(F.current_date(),
                                                F.col("date_naissance")) /
12).cast("int"))
)
sortie.show(5, truncate=False)
```

Étape 4 – Filtrer les personnes de 20 ans et plus

```
sortie_20ans = sortie.filter(F.col("age") >= 20)
print(sortie.count(), sortie_20ans.count())
```

Étape 5 – Exporter les trois fichiers

Spark ne sait pas écrire de fichier Excel : pour ce petit volume, on convertit le DataFrame en DataFrame pandas. Le format XML, lui, est géré nativement depuis Spark 4.0.

```
import os
os.makedirs(SORTIE, exist_ok=True)

# Excel : Spark n'a pas d'export Excel natif, on passe par pandas (petit
volume)
sortie.toPandas().to_excel(f"{SORTIE}/Sortie_Excel.xlsx", index=False)
sortie_20ans.toPandas().to_excel(f"{SORTIE}/Sortie_Excel_20ans.xlsx",
index=False)

# XML : source de données native depuis Spark 4.0
(sortie.coalesce(1).write.mode("overwrite").format("xml")
      .option("rootTag", "personnes").option("rowTag", "personne")
      .save(f"{SORTIE}/Sortie_XML"))

# Bonus : le format de référence en big
datasortie.write.mode("overwrite").parquet(f"{SORTIE}/personnes.parquet")
```

Étape 6 – Vérifier le résultat

Ouvrez les deux fichiers Excel et le contenu du dossier `Sortie_XML` dans JupyterLab. Contrôlez le tri, le format des emails et l'âge minimum dans le fichier filtré.

Questions

1. Pourquoi Spark écrit-il un dossier `Sortie_XML/` et non un simple fichier XML ? Quel est le rôle de `coalesce(1)` ?
2. Ouvrez la Spark UI (http://localhost:4040). Pourquoi aucun job n'apparaît-il après `orderBy()` ou `withColumn()`, mais seulement après `show()` ou `count()` ?
3. Pourquoi `toPandas()` serait-il une mauvaise idée sur un jeu de 500 millions de lignes ?
4. Pourquoi le calcul de l'âge par `months_between` est-il plus juste qu'une division du nombre de jours par 365 ?

Partie 2 : intégration des données de ventes dans PostgreSQL

Votre client vous envoie sa base de ventes. Ces données peuvent éclairer ses performances passées, les tendances du marché et les préférences de ses clients. Votre mission : les extraire, les nettoyer et les charger dans une base PostgreSQL. Créez le notebook `work/partie2.ipynb`.

Attention : le client vous a prévenu que ses fichiers contiennent des factures erronées, des doublons et des enregistrements vides.

Étape 1 – Récupérer les données

Téléchargez les fichiers depuis le dossier du client et placez-les dans `data/`. Vous devez avoir trois fichiers CSV : `Features data set.csv`, `sales data-set.csv` et `stores data-set.csv`.

Étape 2 – Extraire (le E de ETL)

Chargez les trois fichiers. On lit volontairement toutes les colonnes en texte : c'est vous qui déciderez des types, après nettoyage.

```
from pyspark.sql import functions as F
from ws_spark import get_spark, PG_URL, PG_PROPS, LAKE, DATA

spark = get_spark("partie2")

def lire_csv(nom):
    return spark.read.option("header", True).csv(f"{DATA}/{nom}")

features_brut = lire_csv("Features data set.csv")
sales_brut = lire_csv("sales data-set.csv")
stores_brut = lire_csv("stores data-set.csv")
```

```
for nom, df in [("features", features_brut), ("sales", sales_brut),
("stores", stores_brut)]:
    print(nom, df.count(), df.columns)
```

Notez le nombre de lignes de chaque fichier : vous en aurez besoin pour vérifier que votre traitement ne perd aucune donnée.

Étape 3 – Analyser la qualité des données

Avant de transformer, observez. Pour chaque fichier, répondez aux questions suivantes en écrivant les requêtes Spark nécessaires.

1. Affichez une vingtaine de valeurs de la colonne `Date` du fichier features. Que remarquez-vous ? Combien de formats différents trouvez-vous ? Comptez les lignes de chaque format.
2. Le premier nombre d'une date correspond-il au jour ou au mois ? Comment le prouver à coup sûr ?
3. Quelles valeurs non numériques trouvez-vous dans les colonnes `CPI` et `Unemployment` ?
4. Combien de lignes entièrement vides contient chaque fichier ? Combien de doublons ?
5. Exécutez `features_brut.select(F.col("CPI").cast("double")).collect()`. Que se passe-t-il, et pourquoi ?

Indices : `rlike()`, `groupBy().count()`, `filter()`, `F.split()`, `dropna(how=« all")`, `dropDuplicates()`.

Étape 4 – Transformer (le T de ETL)

Complétez le squelette suivant pour produire trois DataFrames propres : `features`, `sales` et `stores`.

```
def unifier_date(col):
    # TODO : renvoyer une colonne de type DATE, quel que soit le format
d'origine.
    # Indices : F.try_to_timestamp(col, F.lit("<format>")) renvoie NULL si
le format
    # ne correspond pas ; F.coalesce(a, b) renvoie la première valeur non
nulle.
    …

def nombre(nom_col):
    # TODO : convertir en DOUBLE en transformant les valeurs invalides en
NULL.
    # Indice : F.expr("try_cast(... AS DOUBLE)")
    …

features = (
    features_brut
    # TODO : supprimer les lignes vides
    # TODO : sélectionner et typer chaque colonne, renommer en minuscules
    # TODO : supprimer les doublons (quelle est la clé d'une ligne ?)
)
```

```
sales = ... # TODO : même démarche (colonnes Store, Dept, Date, Weekly_Sales, IsHoliday)
stores = ... # TODO : même démarche, un magasin ne doit apparaître qu'une fois
```

Exigences :

- les dates sont stockées en type `DATE`, pas en texte ;
- `IsHoliday` devient un booléen ;
- aucune ligne valide n'est perdue en route.

Étape 5 – Contrôler avant de charger

Pour chaque DataFrame, affichez le nombre de lignes et le nombre de dates restées à `NULL`. Ce dernier doit valoir 0. Sinon, affichez les valeurs brutes concernées et corrigez `unifier_date`. Comparez aussi vos comptages à ceux de l'étape 2 et justifiez chaque écart.

Étape 6 – Charger (le L de ETL)

Une fois vos données propres, chargez-les dans la base `ventes` :

```
for nom, df in [("features", features), ("sales", sales), ("stores", stores)]:
    df.write.jdbc(PG_URL, nom, mode="overwrite", properties=PG_PROPS)
```

Bonus : enregistrez aussi `features` en Parquet dans la zone silver du datalake, `f"{LAKE}/silver/features"`.

Étape 7 – Vérifier dans PostgreSQL

Connectez-vous avec `docker exec -it ws-postgres psql -U etl -d ventes`, puis :

- listez les tables avec `\dt` ;
- comptez les lignes de chaque table ;
- affichez la plage de dates de la table `sales` ;
- ajoutez une clé primaire sur `stores(store)`. Si la commande échoue, que cela signifie-t-il ?

Questions

1. Que signifie `mode="overwrite"` ? Que se passerait-il en relançant le notebook avec `mode="append"` ?
2. Dans l'ancienne version Talend de ce workshop, une partie des lignes disparaissait sans message clair. Pourquoi lire les CSV en texte puis utiliser `try_cast` évite-t-il ce problème ?
3. Pourquoi est-il préférable de stocker une date en type `DATE` plutôt qu'en texte `jj-mm-aaaa` ?

Partie 3 : modèle en étoile, datalake et ClickHouse

Cette partie vous fait concevoir un modèle décisionnel en étoile, l'alimenter dans un datalake, puis l'analyser dans ClickHouse. Vous mobilisez tout ce que vous avez pratiqué dans les Parties 1 et 2. Créez le notebook `work/partie3.ipynb`.

Étape 1 – Récupérer les données

Téléchargez la seconde base depuis ce dossier et placez `Customer.csv` et `Product.csv` dans `data/`.

Étape 2 – Comprendre le besoin métier

Votre second client souhaite connaître son chiffre d'affaires (CA) par produit et par client. Objectifs : anticiper l'agencement des produits par région, limiter les pertes et récompenser ses meilleurs clients. Pour aller plus loin, il aimerait aussi ses 10 meilleurs clients et le nombre de ventes par produit (les fichiers ne contiennent pas de quantité).

Étape 3 – Concevoir le schéma décisionnel

Les données proviennent d'un seul métier, la vente : le modèle retenu est donc un schéma en étoile. Sur papier ou dans un outil de votre choix, dessinez votre schéma en précisant :

- le **grain** de la table de faits (que représente une ligne ?) ;
- les mesures ;
- les **dimensions** et leurs attributs ;
- les **clés de substitution** qui relient les faits aux dimensions.

Remarque : la dimension temps est indispensable dans la grande majorité des cas. Faites valider votre schéma par votre tuteur avant de coder.

Étape 4 – Zone bronze : déposer les données brutes

Un datalake s'organise en zones. La zone **bronze** conserve les données telles qu'elles sont arrivées ; la zone **silver** contient les données nettoyées ; la zone **gold** contient les données prêtes à l'analyse, ici votre modèle en étoile.

Avant d'écrire la moindre ligne de code, ouvrez `Customer.csv` et `Product.csv` dans un éditeur de texte et observez-les. Les fichiers se lisent ensuite ainsi :

```
from pyspark.sql import functions as F, Window
from ws_spark import get_spark, LAKE, DATA

spark = get_spark("partie3")
```

```
def lire_csv(nom):
      return (spark.read
            .option("header", True)
            .option("sep", ";")
            .option("encoding", "iso-8859-1")
            .option("escape", '"')
            .csv(f"{DATA}/{nom}"))

customers_brut = lire_csv("Customer.csv")
products_brut = lire_csv("Product.csv")

for nom, df in [("customer", customers_brut), ("product", products_brut)]:
      (df.withColumn("_ingere_le", F.current_timestamp())
      .write.mode("overwrite").parquet(f"{LAKE}/bronze/{nom}"))
```

Questions : à quoi sert chacune des options `sep`, `encoding` et `escape` ? Retirez-les une par une et observez ce que devient le DataFrame (nombre de colonnes, noms avec accents, contenu de `Sales`).

Vérifiez dans la console RustFS (http://localhost:9001) que le dossier `bronze/` est apparu dans le bucket `datalake`.

Étape 5 – Zone silver : analyser et nettoyer

Les deux fichiers ont le même nombre de lignes : trouvez la colonne qui les relie et joignez-les. Analysez ensuite la qualité des données, comme en Partie 2 :

1. Combien de lignes n'ont pas de montant (`Sales`) exploitable ? Ouvrez quelques-unes de ces lignes dans le fichier brut : que s'est-il passé lors de l'export ?
2. Combien de codes postaux sont vides ?
3. Un même `Product_ID` porte-t-il toujours le même nom ?
4. Le format des dates est-il homogène ?

Produisez un DataFrame `ventes` propre et typé. Ne supprimez pas les lignes sans montant en silence : isolez-les dans un DataFrame `rejets`. Écrivez les deux dans la zone silver (`f"{LAKE}/silver/ventes"` et `f"{LAKE}/silver/rejets_ventes"`).

Étape 6 – Zone gold : alimenter les dimensions

Alimentez toujours **d'abord les dimensions**. Ce n'est qu'une fois toutes les dimensions créées que vous construirez la table de faits, qui accueillera leurs clés de substitution et les mesures.

```
def avec_cle(df, nom_cle, ordre):
      # Ajoute une clé de substitution séquentielle : 1, 2, 3...
      return df.withColumn(nom_cle,
F.row_number().over(Window.orderBy(*ordre)))

# TODO : une dimension par entité de votre schéma, par exemple
# dim_client = avec_cle(<DataFrame des clients, sans doublon>, "client_sk",
["<clé naturelle>"])
```

Pour la dimension temps, décomposez la date en jour, mois, trimestre et année. Une clé de la forme `aaaammjj` (par exemple 20261023) est une convention courante. Fonctions utiles: `F.dayofmonth`, `F.month`, `F.quarter`, `F.year`, `F.date_format`.

Indice : comptez le nombre de villes différentes dans lesquelles un même client a été livré.Qu'en concluez-vous sur la place de l'adresse dans votre modèle ?

Étape 7 – Zone gold : construire la table de faits

Partez des lignes de vente nettoyées. Récupérez chaque clé de substitution par jointure avec la dimension correspondante. Agrégez ensuite au grain choisi à l'étape 3. Fonctions utiles : `join()`, `groupBy().agg()`, `F.sum()`, `F.round()`.

Avant d'écrire, effectuez deux contrôles :

1. **Intégrité** : aucune ligne de faits ne doit avoir une clé à `NULL`.
2. **Cohérence** : le CA total calculé sur les ventes nettoyées doit être identique au CA total de la table de faits.

Conseil : faites l'essai d'arrondir le CA à 2 décimales dans l'agrégation, puis sans arrondi. Comparez les deux totaux et concluez sur le bon moment pour arrondir.

Écrivez enfin chaque dimension et la table de faits dans `f"{LAKE}/gold/<nom_de_table>"`.

Étape 8 – Vérifier le datalake

Dans la console RustFS, le bucket `datalake` doit contenir les dossiers `bronze/`, `silver/` et `gold/`. Ouvrez un dossier de `gold/` : quels fichiers y trouvez-vous ?

Étape 9 – Charger l'étoile dans ClickHouse

Ouvrez http://localhost:8123/play (etl / etl). ClickHouse peut lire directement les fichiers Parquet du datalake grâce à la fonction `s3()`. Voici le modèle pour une table : adaptez-le à chacune de vos tables, en choisissant la bonne colonne de tri.

```
CREATE DATABASE IF NOT EXISTS dw;

CREATE OR REPLACE TABLE dw.<nom_de_table>
ENGINE = MergeTree ORDER BY <colonne_de_tri> SETTINGS allow_nullable_key = 1
AS
SELECT * FROM s3('http://rustfs:9000/datalake/gold/<nom_de_table>/*.parquet',
'rustfsadmin', 'rustfsadmin', 'Parquet');
```

Étape 10 – Répondre au besoin métier

Écrivez les requêtes SQL qui donnent :

1. le CA par produit, du plus élevé au plus faible ;
2. le CA par client et par région ;
3. les 10 meilleurs clients ;
4. le nombre de ventes par produit ;

5. pour aller plus loin : l'évolution mensuelle du CA par région.

Questions :

1. Pourquoi faut-il alimenter les dimensions avant la table de faits ?
2. Pourquoi utiliser des clés de substitution plutôt que les identifiants des fichiers sources ?
3. À quoi servent les zones bronze et silver, alors que seule la zone gold sert à l'analyse ?
4. Quelle différence de performance attendez-vous entre une requête directe sur `s3(...)` et une requête sur une table MergeTree ? Pourquoi ?
5. Quelle part des lignes de vente avez-vous dû rejeter ? Que diriez-vous au client sur la fiabilité du chiffre d'affaires calculé, et que lui demanderiez-vous ?

Équivalents Talend

Sur Talend/Cloudera	Dans ce workshop
tRowGenerator	Faker + `spark.createDataFrame()`
tFileInputDelimited	`spark.read.option("header", True).csv()`
tSortRow	`orderBy()`
tMap (expressions, colonnescalculées)	`withColumn()`, `select()`, fonctions `F.*`
tMap (filtre)	`filter()`
tMap (jointures)	`join()`
tUniqRow	`dropDuplicates()`
tAggregateRow	`groupBy().agg()`
tLogRow	`show()`, `printSchema()`
tFileOutputExcel / tFileOutputXML	`toPandas().to_excel()` / `write.format("xml")`
tDBOutput PostgreSQL	`write.jdbc()`
tHDFSOutput / tHDFSInput	`write.parquet("s3a://…")` / `read.parquet("s3a://…")`
`hdfs dfs -ls`, explorateur Hue	Console RustFS
Exécuter le Job	Exécuter les cellules du notebook, suivi dans laSpark UI

Aide-mémoire PySpark

```
df.printSchema() # structure et types
df.show(10, truncate=False) # 10 premières lignes en entier
df.count() # nombre de lignes
df.select("a", "b") # choisir des colonnes
df.withColumn("c", F.col("a") * 2) # ajouter ou remplacer une colonne
df.filter(F.col("a") > 0) # filtrer
df.groupBy(« a").agg(F.sum("b").alias("total"))
df.join(autre, "cle", "left") # jointure
df.dropna(how="all") # lignes entièrement vides
df.dropDuplicates(["cle"]) # doublons sur une clé
df.write.mode("overwrite").parquet(chemin)
```

En cas de problème

Symptôme	Piste
Erreur `S3AFileSystem` ou `NoSuchMethodError`au premier accès `s3a://`	Vérifiez `HADOOP_VERSION` dans`ws_spark.py`, puis redémarrez le noyau
`NoSuchBucket`	Créez le bucket `datalake` dans laconsole RustFS
`CAST_INVALID_INPUT`	Une valeur ne peut pas être convertie :relisez la Partie 2, étape 3
`Connection refused` vers PostgreSQL	`docker compose ps` ; depuis Spark,utilisez le nom de service `postgres`, pas`localhost`
Modification de `ws_spark.py` sans effet	Redémarrez le noyau Jupyter
Noyau qui redémarre seul ou notebook très lent	Allouez au moins 6 Go de mémoire àDocker Desktop
INVALID_PARAMETER_VALUE.CHARSET à lalecture d'un CSV	Spark 4 n'accepte qu'une liste courted'encodages : utilisez iso-8859-1 pour lesfichiers Windows de la Partie 3